

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

The Book of Analogies

38 teaching analogies explaining the architecture — from the Neural Core
to the Big Picture

Author: Renan Augusto Macena

August 2026

The Book of Analogies — Ultimate CS2 Coach

Author: Renan Augusto Macena **Base revision:** in sync with [analogy-book.md](#) (Italian canonical) as of 2026-08-15. **Canonical language:** Italian (see [analogy-book.md](#)). This is the English translation; the Portuguese one is in [analogy-book-pt.md](#) . **Purpose:** To gather in a single volume every pedagogical analogy scattered across the four parts of the *Ultimate CS2 Coach*. The analogies are the pedagogical heart of the book — they make comprehensible to anyone a system that would otherwise require a doctorate in machine-learning engineering.

Introduction

This is the section Renan writes. His voice, his story, the reason behind these metaphors. I cannot replace him here. The paragraph that follows is a placeholder until Renan fills it in — it is not my work, and it must be deleted or rewritten by the human author before publication.

[Placeholder Renan: tell here why you chose concrete analogies — the factory, the school, the jury, the hospital — instead of equations. Tell the first time a friend asked you "but how does it really work?", and you answered with a metaphor that lit up the lightbulb in his eyes. This is the most important lesson of the book, and it must be told in your voice, not mine.]

The analogies in this volume are **canonical**: each one is the reference point for all the variants that appear across the coach books. When a chapter says "*like the jury of three judges*", the entire full story lives here, in entry **§1.2** of this volume. The books can then focus on technical precision — dimensions, constants, invariants — while deep understanding remains preserved in this collection.

This separation is not a compromise — it is a gain. The coach's technical books are dense by necessity: they must cite files and lines, describe mathematical invariants, specify parameters and contracts. If the full-length metaphors were embedded inside that density, the technical reader would find the analogies a slowdown; but the analogies are indispensable for anyone who does not live in the code. By separating them into this volume, we end up with two complementary books: one for those who must *implement*, one for those who must *understand*. The best readers will use both.

How to read this book

1. **Open the chapter of the coach book you are studying** (Part 1A, 1B, 2, or 3). When a technical passage resists you, look the topic up in the **Legend** table below: the *Where it originates* column lists the source chapter of every entry, so it reads backwards too — from chapter to analogy.

2. **Read the canonical analogy** in this volume (the corresponding section).
3. **Return to the book** and continue the technical reading: at that point the metaphor is anchored and the rest of the chapter can talk about `METADATA_DIM=25` , `GLOBAL_SEED=42` , and `input_dim=153` without losing the narrative thread.
4. **Follow the cross-references** (the "*Compare with*" section at the end of each entry): many analogies illuminate each other.

The volume is organized into **ten thematic sections** matching the major blocks of the coach architecture:

- **§1 — Neural Core** (7 entries): the models that do the central thinking — MoE, Superposition, ResNet, Value Critic, GhostEngine.
- **§2 — Memory and Time** (3 entries): how the coach remembers, what it forgets, and the temporal scale of its observations.
- **§3 — Perception and Data** (2 entries): the coach's five senses and the canonical 25-dimensional contract that unifies them.
- **§4 — Strategy and Coaching** (6 entries): the battle plan, the fallback chain of services, the attribution of narrative roles, the distrust toward what the coach asks for on its own.
- **§5 — Knowledge and Retrieval** (3 entries): how the coach knows what it knows — the FAISS library, the COPER diary, the graph of facts.
- **§6 — Training and Orchestration** (4 entries): the school, the orchestra, the exam, the talent scout — how one grows from nothing to master chef.
- **§7 — Analysis Engines** (3 entries): the specialized detectives who see things the other engines don't see.
- **§8 — Data and Ingestion** (4 entries): the post office, the customs officer, the chronicler, the deli counter ticket — how demos arrive, get understood, and get taken in charge exactly once.
- **§9 — Database and Storage** (2 entries): the three-tier archive, the backup vault.
- **§10 — System Overview** (4 entries): the robot that watches videos, the coach's city, the five-layer dashboard, the single palette that colours them.

Legend — where each analogy originates

This table maps the **38 canonical entries** to the chapters of the four books where the technical concept is covered. Useful when a reader wants to go back to the context the metaphor grows out of.

A note on references. Until August 2026 this column carried line coordinates (`1A:329`). They were a promise nobody could keep: adding a single paragraph to a book slid every line after it, and the English and Portuguese editions carried the *Italian* line numbers anyway, which meant nothing in their own files. References are now by **chapter**: they survive revisions and read identically in all three languages.

Entry	Title	Where it originates
§1.1	The Factory with 6 Departments	Part 1A §2 (architecture overview)
§1.2	The Jury of 3 Judges	Part 1A §3 (MoE) · Part 1B §4 (Strategy)
§1.3	The 256-Channel Mixer	Part 1A §3 (SuperpositionLayer) · Part 1B §4 (Strategy)
§1.4	The 7-Story Building	Part 1B §4 (RAP Coach, 7 components)
§1.5	The Coach's Three Pairs of Glasses	Part 1B §4 (Perception)
§1.6	The Hologram of the Better You	Part 1B §4 (GhostEngine) · Part 3 §12.10
§1.7	The Mental Report Card	Part 1A §3 (Maturity Observatory)
§2.1	The Hopfield Album	Part 1B §4 (LTC + Hopfield Memory)
§2.2	The Slow Mentor (EMA)	Part 1A §3 (EMA module)
§2.3	The Three Lenses of the Chronovisor	Part 1B §4 (ChronovisorScanner)
§3.1	The Coach's 5 Senses	Part 1B §5 (Data Sources) · Part 1A §2
§3.2	The Sensor's 25 Questions	Part 1A §2 (25-dim contract) · Part 3 §12.6
§4.1	The Hospital and the Reception Desk	Part 2 §5 (Coaching Services)
§4.2	The Detective of 5 Errors	Part 1B §4 (Pedagogy — Causal Attribution)
§4.3	The Commentator of Value	Part 1B §4 (Pedagogy — Value)
§4.4	The CS2 Chess Tree	Part 2 §7 (Analysis Engines — game theory)
§4.5	The Poker Face of Deception	Part 2 §7 (Deception Index)
§4.6	The Distrustful Librarian	Part 2 §5 (CoachingDialogueEngine — agentic phase)
§5.1	The FAISS Library	Part 1B §5 (Vector Index) · Part 2 §6
§5.2	The Textbook + Diary	Part 2 §6 (Knowledge and Retrieval)
§5.3	The Graph of Facts	Part 2 §6 (Knowledge Graph)
§6.1	The Coach's School	Part 1A §3 · Part 3 §10 (training regime)
§6.2	The Training Orchestra	Part 2 §12 (Training Pipeline)
§6.3	The Exam with Answers on the Back	Part 2 §13 · Part 3 §11 (loss functions)
§6.4	The Talent Scout of Roles	Part 2 §7 (Role Classifier)
§7.1	The Squad of Detectives	Part 2 §7 (the 11 analysis engines)
§7.2	The Mood Ring	Part 2 §7 (Momentum)
§7.3	The Driving Instructor	Part 2 §7 (Movement Quality)
§8.1	The Post Office of the Letter	Part 3 §12.6 (Ingestion Pipeline)
§8.2	The Demo Customs Officer	Part 1B §5 (Demo Format Adapter)

Entry	Title	Where it originates
§8.3	The HLTV Sports Chronicler	Part 1B §5 (HLTV module) · Part 3 §12.24
§8.4	The Deli Counter Ticket	Part 3 §12.6 (atomic task claim)
§9.1	The 3-Tier Archive	Part 2 §11 · Part 3 §9 and §12.9
§9.2	The Backup Vault	Part 2 §11 · Part 3 §12.9
§10.1	The Robot that Watches Video	Part 1A §1 (executive summary)
§10.2	The Coach's City	Part 3 §12 (complete program logic)
§10.3	The Five-Layer Dashboard	Part 3 §12.5 · Interconnection Map
§10.4	The Single Palette	Part 3 §12.5 (design tokens and theme relay)

§1 — Neural Core

§1.1 The Factory with 6 Departments

Think of the system as a **large factory with six connected departments**. The first department (Ingestion) is the **mailroom**: it receives the raw match recordings — the `.dem` files — and sorts them by map, team, and date. The second (Processing) is the **workshop**: it takes the recordings and measures every tick — position, weapon, health — turning raw bits into a canonical 25-dimensional vector. The third (Training) is the **school**: it teaches the AI's brain by showing it thousands of examples, progressing from pistol rounds to full buys. The fourth (Knowledge) is the **library**: it stores tips, past experiences, and professional references so the coach can consult them. The fifth (Inference) is the **brain**: it combines what the AI has learned with what the library knows to produce advice. The sixth (Analysis) is the **investigative squad**: it runs special inquiries — *"is this player struggling?"*, *"was that a good position?"* — using eleven analytical engines. All six work in parallel: when you open the desktop app, a Scanner daemon scans folders, a Digester analyzes demos, a Teacher updates the models, and a Pulse watches over the overall health. The six departments of this factory correspond exactly to the subdirectories of `backend/`: `ingestion/`, `processing/`, `nn/`, `knowledge/`, `services/`, `analysis/`. The factory is not a wasteful company — each department has a precise purpose, and no department does another department's work. A demo arriving at the mailroom is never processed in the workshop until it has been registered; a model is never shipped to production until the school has promoted it; advice is never given without the library and the brain first agreeing. The organization guarantees that a failure in one department — a corrupted demo, a training crash, a lookup miss — does not paralyze the others.

Compare with: §6.1 (The Coach's School — the third department seen from inside), §9.1 (The Archive — how the fourth department preserves memory), §10.2 (The City — the same factory seen as urban infrastructure).

§1.2 The Jury of 3 Judges

Some of the coach's models work like a **jury of three judges at a talent show**. The LSTM reads the game sequence as if reading a story — it grasps what happened step by step, remembering the important moments — and produces a summary in 128 numbers, its "opinion". At that point, three experts examine the summary and each one assigns a score. But the judges are not equally competent in every situation: an aim expert judges duels best, an economy expert judges eco rounds best, a positioning expert judges defensive setups best. For this reason a **gating network** — like a moderator — decides how much to trust each judge at this specific moment: *"For this tick, Judge 1 weighs 60%, Judge 2 weighs 30%, Judge 3 weighs 10%"*. The final score is the weighted average of the three opinions. In the RAP Coach the same metaphor extends to **four generals** seated in the war room (`SuperpositionLayer + MoE`, top-k=2), where a softmax gate picks the two most relevant ones and lets them debate with each other before producing the collective decision. This is how the network avoids being a mediocre generalist: it specializes, and specialization only activates when it is needed. The Mixture-of-Experts mechanism is one of the most elegant in modern machine learning — instead of a monolithic network that tries to know everything, many small specialists are trained and the routing is learned contextually. A single general who is an expert in mid-round would never be called during a 1v1 clutch; a single clutch expert would never be called during an eco setup. The gate is not a fixed parameter — it is a small network that trains alongside the rest, learning to route cases well.

Compare with: §1.3 (The Mixer — how each judge's intensity is modulated), §6.2 (The Orchestra — how the judges are trained together).

§1.3 The 256-Channel Mixer

Imagine an **audio mixer with 256 channels and thousands of potentiometers** that move by themselves based on context: when you play a jazz passage, only the right channels rise; when you play rock, different channels take over. The `SuperpositionLayer` works this way. Every neuron is a channel; for every game tick, the context — player role, map, round phase — automatically determines which neurons should light up and which should stay silent. An **L1 sparsity** term in training pushes the model to use the fewest possible neurons to arrive at the right answer — like a good sound engineer who gets a clean sound with a few active channels rather than cranking everything to maximum. The result is a model that **reuses the same brain** for ten roles and seven maps without falling into the trap of being "good at everything, expert at nothing". Every neuron is like a dimmer, and the context decides the brightness: when an AWPPer plays on Nuke on the T side, exactly the neurons needed for that specific problem light up. The beauty of the mechanism is that sparsity is not rigidly imposed — it emerges naturally from optimization. Training shows the model that using 80 out of 256 neurons for this role is "rewarded" (lower loss) compared to using all of them; and the model, incentivized, automatically learns to specialize. L1 sparsity is gentler than L0 (which would count active neurons discretely and be non-differentiable): it penalizes the sum of the absolute values of the weights, pushing toward zero those that are not needed without forcing them to be exactly zero. It is the perfect compromise between cleanliness and trainability.

Compare with: §1.2 (The Jury — the judges that the mixer "illuminates"), §6.3 (The Exam — L1 sparsity seen as an exam criterion).

§1.4 The 7-Story Building

The RAP Coach — the system's main neural model — is a **seven-story building** where each floor has a specialized function. The first floor is **Perception**: three convolutional streams process the player's view, the minimap, and the movement delta (see §1.5). The second is **Memory**: LSTM + Hopfield keep track of recent context (see §2.1). The third is **Strategy**: SuperpositionLayer and Mixture-of-Experts choose the type of advice (see §1.3). The fourth is **Pedagogy**: the Value Critic estimates "how good is this situation" and a Skill Model adapts the advice to the player's level. The fifth is **Positioning**: it predicts where the player should move (see §1.6). The sixth is **Causal Attribution**: it explains *why* something went wrong (see §4.2). The seventh is **Output**: it assembles all the results into a single dictionary ready for the application layer. Between the floors there are **identity elevators** (ResNet-style skip connections) that let information skip floors and arrive straight at the top when needed — an essential trick for training deep networks without the gradient dissolving. Without the elevators, at the depth of the sixth floor the signal would already be too faded to be useful. Every floor has well-defined inputs and outputs — a contract that allows a floor to be swapped with a better variant without breaking the rest of the building. This is how the project was able to evolve: when the Perception floor was upgraded (3 streams instead of 2), the other floors did not have to be rewritten; when the Memory floor received the addition of Hopfield alongside the LSTM, the Strategy floor did not notice the difference. This modularity is the secret to how the RAP Coach was able to reach a complexity that would have been unmanageable in a monolithic architecture.

Compare with: §1.5 (The Three Pairs of Glasses — detail of floor 1), §2.1 (The Album — detail of floor 2), §4.2 (The Detective — detail of floor 6).

§1.5 The Coach's Three Pairs of Glasses

The RAP Coach's Perception layer is like **three pairs of glasses** worn simultaneously. The **first pair** (ventral stream) shows *what* the player sees — their first-person perspective — processed by a lightweight ResNet that extracts 64 features from the image. The **second pair** (dorsal stream) shows the **aerial minimap** — where everyone is — processed by a simpler network into 32 features. The **third pair** shows the **movement delta** — who is moving and how fast — into another 32 features. All three views are concatenated into a single 128-number vector: "*Here is everything I can see in this instant*". The separation is inspired by the human brain: the ventral stream recognizes *what* things are, while the dorsal tracks *where* they are. Crucially: all the inputs are built by the **TensorFactory** respecting the **NO-WALLHACK** principle, meaning the coach only sees what the player sees. There is no external camera that leaks information hidden behind obstacles. If a player cannot see an enemy behind a corner, the coach cannot penalize them for not anticipating it. It is a driving test where the instructor judges only what the student could reasonably have seen. NO-WALLHACK is not just an ethical rule — it is a rule of *pedagogical fairness*. A coach who knew where the hidden enemies were would teach the player strategies impossible to

replicate in reality, where enemies stay hidden. The coach must live in the same fog of war as the player — and it is from that shared fog that trust emerges: *"my coach advises me on a play that is actually possible, not a play that requires powers I don't have"*.

Compare with: §3.1 (The 5 Senses — how the data reaches the three pairs of glasses), §10.1 (The Robot that Watches Video — end-to-end perception).

§1.6 The Hologram of the Better You

The **GhostEngine** produces a **transparent hologram** on the tactical map that shows *where you should have been*. At every tick, it asks the RAP Coach: *"Given this exact situation, where SHOULD the player be?"*. The answer is a small positional delta — for example *"five units to the right, three forward"* — scaled by $\times 500$ into CS2 map coordinates. The result is an *"ideal you"* ghost drawn on the minimap, running alongside the real player. If the ghost is far away, you know you were in a bad position; if it is close to you, your positioning was correct. Inference happens in real time over a **window of the last 32 ticks** (the **sequence_length** of the state reconstructor and the TensorFactory trajectory window) — a "circular tape recorder" — and the inference engine keeps the recurrent memory state between consecutive ticks (the famous **hidden_state** from the seventh output of the RAP Coach), so it does not "forget" what happened five seconds ago when it evaluates you now. The ghost is not a final judgment of your play: it is a visual reference, a "better you" that appears where the coach thinks you should have been, and disappears when you coincide with it. The beauty of the ghost is that it is not prescriptive in an authoritarian way — it is suggestive in a gentle way. It doesn't tell you "you were wrong"; it shows *where it would be better to be*, and leaves the choice to you to follow it. After a few hours of practice with the ghost visible, players report feeling its presence even when it is turned off — they have internalized the pattern, and no longer need the hologram because now they already think like it. The ghost is therefore a transitional teacher: it helps you grow and then becomes unnecessary.

Compare with: §3.2 (The 25 Questions — the input that feeds the hologram), §4.3 (The Commentator of Value — judgment on "how good" the position is).

§1.7 The Mental Report Card

While the coach trains, an **Introspection Observatory** keeps a report card of its mental health. It does not look only at total loss — that would be like judging a student by the final average alone. Instead it measures **five signals** that compose a *Conviction Index* from 0 to 1: the entropy of beliefs (is it still hesitating?), the specialization of the gates (is each expert learning its own role?), conceptual focus (are the 16 VL-JEPA concepts differentiating?), value accuracy (are the Value Critic's estimates improving?), and role stability (are the predictions oscillating or coherent?). These five signals define four maturity states: **DOUBT** (<0.30 — the model is still asking itself who it is), **LEARNING** ($0.30\text{--}0.60$ — it is studying), **CONVICTION** (>0.60 stable for 10+ epochs — it has formed a view), **MATURE** (>0.75 stable for 20+ epochs — it can go to production). A sharp drop $>20\%$ triggers the **CRISIS** state, because a sudden loss of confidence is more suspect than a single error. A doctor never judges a patient by a heartbeat at one instant: they look at the trend. The *Conviction Index* does the same thing for the AI's brain, and it feeds

TensorBoard for real-time dashboards. The Observatory is not just instrumentation — it is philosophy: a model that is learning badly must be stopped before it becomes a model that has learned badly. At the first CRISIS manifestation, training is paused and the recent epochs are examined: is it a bug in the dataset? L1 sparsity too aggressive? A drift in the preprocess statistics? This self-diagnostic discipline is what distinguishes an engineered ML system from an experiment that "we hope works".

Compare with: §6.3 (The Exam — the individual losses the Observatory measures), §6.2 (The Orchestra — the callbacks that write on the report card).

§2 — Memory and Time

§2.1 The Hopfield Album

The RAP Coach's Memory layer is a **photo album of famous plays**. The Hopfield network is the associative part: during training it has memorized a number of recurring patterns — "*fast push through Mirage mid*", "*Inferno B retake with a 4-utility stack*", "*1v2 clutch with a pistol at round 12*" — and, given a new situation, it retrieves the most similar pattern, like when a Major play comes to mind after you see a fragment of the current one. Next to the album there is a **living LTC brain** (Liquid Time-Constant network): it is not a photo, it is an organism that adapts to the rhythm of the game — when the match speeds up, the brain follows; when it slows down, it relaxes. A crucial detail: **Hopfield is bypassed for the first two training forward passes** (invariant `NN-MEM-01` in `rap_coach/memory.py:74-78`), because an albumist who has not yet memorized anything cannot pick meaningful photos — they would pick at random, spoiling the first gradients. During those passes only the LSTM drives. With 153 input dimensions (128 perception + 25 metadata) it outputs a 256-dimensional hidden state plus a 64-dimensional belief vector — "*what I think is happening*" — that feeds the rest of the 7-story building. If the hardware is modest there is `RAPMemoryLite`, a backup generator with simpler fuel that works when the main building cannot load the full Hopfield. The LSTM + Hopfield synergy is valuable: the LSTM manages the *temporal* chronology (what happened tick-by-tick in the recent seconds) and Hopfield manages the *associative* memory (which similar pattern was seen in other matches). Together they cover two complementary dimensions: "what recently happened to me" and "what happens when the game is in this generic situation". The coach draws on both to build an understanding that is simultaneously personal and universal.

Compare with: §1.4 (The Building — the floor where the album lives), §2.3 (The Three Lenses — how to choose *when* to remember), §5.2 (The Textbook + Diary — external memory that complements the internal one).

§2.2 The Slow Mentor (EMA)

When JEPA trains, it pairs every encoder with an **"older brother" EMA** (Exponential Moving Average) that absorbs the younger brother's knowledge *very slowly*. The younger brother — the online encoder — runs the experiments: it tries, it errs, it corrects, it changes weights every step. The older brother — the target encoder — is the zen mentor: at every step, 99.6% stays as it was, only 0.4% updates toward what the younger one has discovered. This "slow target" prevents the system from **collapsing** into a trivial solution ("everything the same, everything with zero embedding, solution accepted"). In JEPA the momentum starts at **0.996** and follows a **cosine schedule** toward **1.0** during training (`backend/nn/jepa_train.py:353`): at the start the target encoder tracks the online encoder quickly, but as the network matures, the target fossilizes and becomes a stable reference — like a mentor who is curious about every idea of the student at the beginning and then, once they have seen enough, becomes firm. The RAP Coach uses a separate variant with decay 0.999 (`backend/nn/ema.py:39`). Implementation golden rule: `apply_shadow()` MUST `.clone()` the shadow weights (invariant `NN-16` in `ema.py:79`) — otherwise aliasing causes corruption when the mentor's weights are applied/restored during inference. The older-brother metaphor also explains another technical rule: during the EMA update, the target encoder MUST have `requires_grad=False` (invariant `NN-JM-04` in `jepa_model.py:323-331`). If it were still trainable, the optimizer would try to change it too, and the slow mentor would become a confused mentor. Separating the roles is essential: the online encoder learns from gradients, the target encoder learns only from EMA. Breaking this rule produces a training that looks normal but collapses silently after a few hundred epochs.

Compare with: §6.3 (The Exam — how to measure whether the mentor is learning well), §2.3 (The Lenses — the other temporal dimension of training).

§2.3 The Three Lenses of the Chronovisor

The `ChronovisorScanner` hunts for the important moments of a match using **three different magnifying lenses**. The first lens is the *micro zoom* (tick-by-tick scale): it sees the single duel, the single flash, the single jump. The second is the *standard zoom* (round scale): it sees the economic trajectory of the round, the rhythm of the engagements, the positioning close to the bomb timer. The third is the *macro zoom* (match scale): it sees the inter-round momentum, the overall tactical map, the patterns that repeat every six rounds. Without the three scales, the coach would risk getting lost: too much micro and it misses the war of economies; too much macro and it misses the decisive clutch kill. It is the same principle as Google Maps: at street level you don't grasp the topography of the region, at continent level you don't see the street. The three lenses produce three parallel streams of highlights that inference combines. Implementation-wise the Chronovisor decides what to send to the GhostEngine and what to save to the Experience Bank (see §5.2) for future consultation. An important moment is such only if it qualifies on *at least one* of the three scales — a generous criterion because the coach's memory must be inclusive: it is better to remember too much than to forget a moment that in hindsight will reveal itself as a lesson. The three lenses also have the function of calibrating *the language* of the coach. A comment on the micro scale ("your aim was 3 units to the left of the target") sounds different from a comment on the macro scale ("you spent your grenade too early"). The coach chooses the level of detail according to the active lens: it reports granular details only when needed, and uses abstract language for high-level insights.

Compare with: §7.2 (The Mood Ring — another multi-scale, inter-round metric), §5.2 (The Diary — where the moments the lenses find are saved).

§3 — Perception and Data

§3.1 The Coach's 5 Senses

The external data sources are the **five senses** of the AI coach. **Sight** is the `DemoParser` that reads every tick of the `.dem` file through `demoparser2` (version 0.40.2), preserving the full game state — positions, health, equipment, view angles, 128 times per second, with no decimation (tick decimation is a forbidden invariant in CLAUDE.md, because losing even a single one would break the continuity that the LSTM memory requires). **Hearing** is the `EventRegistry` that recognizes all CS2 events — kills, flashes, utility, bomb plant, round end — mapping them to tactical semantics. **Touch** is the `TradeKillDetector` that feels when a teammate falls right after another and the coach assesses "we're in trade window". **Smell** is the `SteamLocator`, the bloodhound that sniffs among Steam's subfolders to find CS2 demos automatically without asking the user for help. The **sixth sense** — the coach's telepathy — is HLTV data: received through a dedicated scraper with Playwright and FlareSolverr, it lets the coach compare against living professional players, updated weekly. Sacred rule of the whole sensory system: the **NO-WALLHACK principle** applies. The coach processes only what the player could see or hear at that moment. Never surveillance cameras, never privileged information. The driving test applies to all senses: one is judged only on what the student could reasonably perceive. Sensory coherence is the basis of **coaching credibility**: if the coach knows something the player did not know, its advice becomes unexecutable — "you should have gone for the retake because there was only one enemy there" sounds useful but is useless if the player had no way of knowing there was only one enemy. A coach that respects the five senses produces advice the player can truly follow: not "you were wrong because you didn't know X", but "you were wrong because you misinterpreted Y, which was visible".

Compare with: §1.5 (The Three Pairs of Glasses — how sight becomes a tensor), §8.3 (The HLTV Chronicler — the sixth sense seen up close), §10.2 (The City — where the senses collect data).

§3.2 The Sensor's 25 Questions

Every game tick is translated into a **standard questionnaire of 25 questions**, always the same, always in the same order. The vector has 25 dimensions and the constant `METADATA_DIM = 25` in `backend/processing/feature_engineering/vectorizer.py:32` is the sole authority: if you change the number of questions without updating `FEATURE_NAMES`, `METADATA_DIM`, and the `input_dim` of all models simultaneously, the whole system explodes with matrix-multiplication errors — and rightly so, because any model trained on one version of the questionnaire cannot understand a different version. The questions are: *How much health? How much armor? Helm? Defuser? Equipment value? Crouching? Scoped? Flashed? How many enemies visible? Position x/y/z? View direction (sin/cos of yaw + pitch)? Z penalty for multi-level maps? Estimated KAST rating? Map ID? Round phase? Weapon class? Time in round? Bomb planted?*

Teammates alive? Enemies alive? Team economy? — 25 numbers, all normalized between -1 and +1 or 0 and 1. It is the *canonical contract*: if you have 128 sensors attached to the player reading these 25 quantities simultaneously 128 times per second, you have a complete description of the game state from which all coaching can be derived. The **FeatureExtractor** is the **universal translator** — the official dictionary — that the whole system consults: training, inference, RAG, COPER. If any one of them used a different dictionary, internal communication would break silently. The 25-dimensional contract was not chosen at random — it is the result of iteration: we started from a smaller vector (12 dimensions) that was losing key information; we moved to an intermediate one (19) that was still missing some normalizations; we reached 25 only when every single question turned out to be *necessary* and *non-redundant*. A 26th question has been debated more than once (for example "time elapsed since last kill"), but every proposal has been rejected either because it was derivable from the 25 existing ones or because the derivation was better left to the models themselves.

Compare with: §1.5 (The Three Pairs of Glasses — the visual questions don't enter here, they enter in the 64×64 tensors), §9.1 (The Archive — where the 25 numbers are stored).

§4 — Strategy and Coaching

§4.1 The Hospital and the Reception Desk

The **Coaching Services** subsystem is like a **hospital's reception desk**. It takes the reports from the specialist doctor (RAP Coach, §1.4), the notes from the nurse (analysis engines, §7.1), the lab results (RAG + COPER, §5) and turns them into a clear report for the patient (the player). It also has a backup plan for every scenario: if the COPER specialist is unavailable because the models are not yet mature, it routes you to the hybrid doctor (**Hybrid**); if the hybrid doctor is absent, it hands you a pre-printed leaflet (**RAG**); and if all else fails, it at least gives you your basic vital signs (**Base**). The patient (player) **ALWAYS** leaves with something useful, never empty-handed. This is the heart of the *4-level fallback chain* (COPER → Hybrid → RAG → Base): there is never a "silence" of the coach. Like choosing between fine-dining pizzeria, trattoria, bakery, bread and water: it depends on how mature the kitchen is today. Implementation-wise this is reflected in **CoachingService** which reads the coach maturity state (CALIBRATING/LEARNING/MATURE) and chooses which pipeline to activate. The **OllamaCoachWriter** (local llama3.1:8b, optional) is the **polyglot receptionist** who translates the dry language of the models into motivating, human Italian. The fallback chain is a deliberate architectural choice: it is easy to build a system that works wonderfully when everything is ready and blows up when something is missing. It is much harder to build a system that degrades gracefully — a worse piece of advice is always better than no advice, especially for a user who is learning. The coach is not perfect on the first demo; but it never leaves the player without a word. And it is precisely graceful degradation that grows the player's trust: even when the coach is young and lightly trained, there is always something useful to say.

Compare with: §5.2 (The Textbook + Diary — the two sources of knowledge the reception integrates), §7.1 (The Squad of Detectives — the reports that arrive at the reception).

§4.2 The Detective of 5 Errors

The RAP Coach's **AttributionHead** is the **causal detective**: after every decision, it does not just say "you were wrong", but answers *"why you were wrong"*. It produces a 5-dimensional vector — five categories of blame — that reads like a school report card: **Positioning** (were you in the right spot?), **Timing** (did you peek at the right moment?), **Aim** (was the crosshair where it should have been?), **Utility** (grenade wasted or never used?), **Economy** (did you buy at the right time?). Every decision gets a breakdown — for example 60% timing, 25% aim, 15% positioning — and the interface shows the player the dominant categories with specific recommendations. It works alongside a latent skill model that describes the player across five subjects with Z-score and percentile: *"you are at the 72nd percentile in aim among pros, at the 34th in economy"*, and the composite training loss is a **4-grade report card** (MSE for position, MSE for value, Cross-Entropy for strategy, KL for attribution) weighted together (`rap_coach/trainer.py:27` for the Z-axis weights). The detective does not point the finger to blame — it points the finger to teach: every error is classified into a precise subject, so the student knows exactly what to train on in the next DM session. The detective's pedagogy is radical in its gentleness: it does not accept the "bad player" narrative — for the detective there is no bad player, there is a distribution of errors across 5 subjects, and each subject is improvable with targeted practice. This framing is liberating for the user: instead of hearing "you lost because you're bad", they hear "you lost 40% of the duels because of timing, 30% because of aim, 20% because of utility, 10% because of positioning — so work on timing this week". It is a tutor's advice, not a judge's verdict.

Compare with: §4.3 (The Commentator of Value — the other half of the judgment: not "why it went wrong" but "how good is it now"), §6.3 (The Exam — the losses the detective minimizes).

§4.3 The Commentator of Value

The RAP Coach's **Value Critic** is a **live sports commentator** that at every instant tells you *"You're now at 72% advantage"* or *"The team is behind, 34% chance of winning the round"*. Unlike the win probability of the **WinProbability** engine (which only looks at the round, §4.4), the Value Critic looks at the **specific moment** — the single decision — and estimates how good it is from the coaching point of view: *if I keep behaving this way, how does the round end?*. It is the heart of the RAP Coach's *pedagogy*: without a value estimate, the model wouldn't know whether a piece of advice is risky or prudent, aggressive or passive. The Value Critic is trained with MSE against a temporally-discounted return target (classical RL value function) and at inference produces a number between -1 and +1 that combines with the causal attribution (§4.2) and with the advice probability vector to produce the final advice *"push with 0.72 confidence, because value is high, timing is dominant, economy is stable"*. The commentator does not replace the detective: they work together. The detective says *"you got the timing wrong"*; the commentator says *"and that timing will cost you the round"*. Together they become a pedagogical chronicle: the diagnosis + the consequence. The difference between an experienced commentator and a rookie is exactly the difference between a well-calibrated value function and a noisy one. An experienced commentator reads the match and can say *"now it's a turning point"* while a novice yells *"goal!"* every time the ball comes near the net. A well-trained Value Critic distinguishes pivotal moments from neutral ones; a badly trained one throws out advice at random.

Compare with: §4.2 (The Detective — the "why" half), §4.4 (The Chess Tree — the win estimate at round scale).

§4.4 The CS2 Chess Tree

For complex strategic scenarios — *"Eco-force CT on Inferno 1v3, bomb planted, 18 seconds"* — the coach explores an **expectiminimax game tree** three levels deep, like a chess engine. At every node, one of the players (or chance, for probabilistic nodes) has a move; the tree estimates the expected value of each one and picks the best. Alongside it there is a **Bayesian danger estimator** (**BeliefModel**) that computes *"how likely am I to die now?"* given three signals: position (am I in a zone with historically high lethality?), equipment (AWP vs pistol?), and state of the enemies (have I heard them? have I already seen them?). The formula is classical Bayesian: prior from pro stats, likelihood from recent ticks, posterior from both. An automatic calibration keeps the goodness of the prior in check to avoid overconfidence. Together, tree + estimator provide a **strategic evaluation** that the coach communicates as *"survival at 23%, push advised only if you have information on the second enemy"*. Three levels are not as deep as a Top-10 FIDE chess engine, but they are enough to cover *"direct duel + trade + follow-up"*, which is CS2's tactical core. Going deeper would become unreliable, because the variance of the transition model grows exponentially with the number of simulated ticks. The choice of three levels is a compromise calibrated empirically: at level 1 the tree does not capture consequences; at level 5 the noise of the transition model (the difficulty of predicting what the enemies will do 3 seconds from now) makes the evaluation worse than the stand-alone Bayesian one. At level 3 the *"duel → trade → follow-up"* pattern is captured — the dominant tactical pattern — and the branches are pruned with alpha-beta pruning adapted to the probabilistic context.

Compare with: §4.5 (The Poker Face — another probabilistic measure, on bluffing), §7.1 (The Squad of Detectives — Win Probability is one of the detectives).

§4.5 The Poker Face of Deception

The **DeceptionIndex** is a **poker-face score**: it quantifies how unpredictable you are compared to the average professional player on the same map and the same side. It counts flash fakes (did you simulate an opening by throwing a flash without pushing?), fake sites (did you disappear from radar toward B only to come back to A?), walk/run mixing (do you alternate silent and loud movement to confuse?), pre-utility tosses (do you throw grenades before moving to confuse the timing?). Every entry has a weight calibrated on how often pros do it in that situation, and the final index is a z-score standardization: 0 = you play exactly like the pro median, +2 = you are clearly more deceptive than the average pro, -2 = you are completely readable. A low deception index is not automatically bad — there are roles (a pure anchor, a disciplined lurker) where predictability is functional — but a chronically low index for an entry fragger is a red flag: you are being read, and that is the most likely explanation for your lost duels. The coach communicates it with a 5-axis radar chart and with the voice *"today you are playing too linearly: try switching the side of the flash on Mirage mid when the CT has read you twice in a row"*. The Deception Index is one of the subtler indicators of the coach: once internalized, the player manages to actively

govern their own predictability — becoming more readable at some moments (to build patterns that they will later break) and less readable at others (to maximize the advantage of the break). It is the strategic mind that emerges from mechanical gameplay, and the coach provides the lens to see it.

Compare with: §7.2 (The Mood Ring — deception + momentum together tell the personality of the round), §4.2 (The Detective — the 5 subjects of which deception is a meta-level).

§4.6 The Distrustful Librarian

Since August 2026 the coach can go and fetch the data itself: ask it *"how did I play the fifth round on Mirage?"* and it no longer waits for someone to place the right context in front of it — it calls one of its four tools and queries the database. The way it is allowed to do so, however, is that of a **closed-stack library**. In such a library the reader does not walk among the shelves: they fill in a request slip and hand it to the librarian, who checks it against the catalogue and goes to fetch the volume. If the slip names a title that is not in the catalogue, the librarian does not start guessing, does not open the stacks to show what is there, and does not take offence: they hand the slip back saying *"that title is not catalogued, consult the list"*. The coach is the reader, and it must be treated like any other reader — not out of distrust, but because what it writes on the slip is **generated text**, not a value picked from a menu: it can be plausible and non-existent at the same time. So every match name the model produces is resolved against a whitelist built by querying the database, round numbers must be integers within a bounded range, team-name fragments are stripped of every character that serves no purpose, and even the answer coming back from the stacks is cleaned and shortened before delivery — because that text, once delivered, becomes part of what the coach reads. There is one last courtesy, and it is the most effective: **the list of available volumes is posted at the library entrance**. Without that list the reader asked for invented books and every request was a wasted trip; with the list in front of it, it asks for what exists.

Compare with: §4.1 (The Hospital and the Reception Desk — the service chain this librarian works in), §8.2 (The Customs Officer — the same distrust applied to files instead of words), §5.1 (The FAISS Library — the other library, the open-stack one for similarity).

§5 — Knowledge and Retrieval

§5.1 The FAISS Library

The tactical knowledge base is a **library** organized by concepts, not by keywords. When the coach wants to know *"how to play the CT retake on site B of Inferno"* it doesn't do a full-text search: it asks **FAISS** (Facebook AI Similarity Search) for the document whose 384-dimensional embedding is closest to the query in cosine space. The 384 numbers per document are produced by a Sentence-BERT model that understands semantics: it knows that *"B-side retake"* and *"site B recovery"* are the same thing even if the words are different. The library contains Coach Book v4: 502 tactical entries in 8 JSON files (7 active maps + general) with 13 categories — positioning, utility, economy, aim_and_duels, mid_round,

retakes_post_plant, anti_strat, role_play (entry/awp/support/lurk/igl), clutch_play. Every entry is a prescriptive paragraph written to feel like *"your coach explaining what to do now"*. The search returns Top-K entries (typically $K=5$) with a contextual boost of $1.2\times$ for entries that exactly match the player's map/side and a deduplication with threshold 0.85 to prevent five nearly-identical paragraphs from appearing in the same piece of advice. It is the equivalent of a librarian who doesn't hand you five copies of the same book even if they are all relevant. The choice of 384 dimensions is not random — it is the standard Sentence-BERT footprint, which offers the best compromise between semantic precision and search speed. A larger embedding would capture subtler nuances but slow down the search; a smaller one would be fast but would lose crucial semantic distinctions. FAISS is configured in "flat" mode (exhaustive search) because 502 entries don't require more complex ANN (Approximate Nearest Neighbor) structures; for much larger knowledge bases (thousands of entries) one would switch to HNSW or IVF, but at that point a small search error would be paid in exchange for speed.

Compare with: §5.2 (The Diary — experiential knowledge that complements the library), §5.3 (The Graph — the connections between entries).

§5.2 The Textbook + Diary

The coach has **two sources of knowledge** that it uses in parallel, like a student studying for an exam. The **textbook** is the RAG (§5.1): public, consolidated knowledge, written by human coaches. The **personal diary** is the COPER **ExperienceBank**: every time the coach gives advice during a match, it writes an entry into the diary — *"Dust2, eco round T-side, player in tunnels B with 60 HP and a Deagle. I told him to hold the angle. He survived and got 2 kills. The advice WORKED"*. Later, when a similar situation comes up, the coach consults both the textbook (what do coaches say in general?) and the diary (what happened last time this specific player was in this situation?). The result is a hybrid piece of advice that speaks with authority *and* with personal memory. KT-01 takes the diary to the next level with **CRUD semantics** (UPDATE/DISCARD/KEEP), **prioritized replay** (high-impact situations are revisited more often), and **TrueSkill integration** (the weight of every entry grows with the historical accuracy of the coaching). An EMA feedback loop ($new_score = 0.7 \times old + 0.3 \times outcome$) makes good advice become more reliable, bad advice attenuate, and after 90 days without validation the confidence in an entry decays by 10%: like a weather forecast that loses reliability as you move further from the moment it was made. The textbook + diary combination is powerful because the two systems cover complementary knowledge spaces. The textbook is general, impersonal, stable — it tells you *what the world's coaches have learned*. The diary is specific, personal, evolving — it tells you *what has worked for this player*. A coach who had only the textbook would give generic advice; one who had only the diary would be blind in the face of new situations. Together, it becomes a tutor who knows both the theory and you.

Compare with: §4.1 (The Hospital — the reception that integrates textbook + diary), §5.1 (The Library — the textbook), §2.3 (The Lenses — what ends up in the diary).

§5.3 The Graph of Facts

Beyond FAISS and the ExperienceBank, there is a **knowledge graph** that links the facts of Coach Book v4 to each other via semantic relations — *"this entry on Mirage mid is the counter-strat of that entry on A rush with flash over"*, *"this pro tactic is the evolution of an older technique"*. The graph enables **multi-hop reasoning**: faced with a specific question, the coach can explore a path — concept A → relation → concept B → relation → concept C — to build a composite answer rather than returning a single paragraph. It is like a chess expert who explains a move by referring not only to that move, but to the opening system it belongs to, to the typical strategic plan, to the endgame it leads to. The graph is dense in spots where CS2 gameplay is richest (defaults on Mirage mid, retake on Inferno site B, eco management) and sparser in marginal areas (secondary metas, obsolete strats). In the implementation the graph is in memory — not persisted to a dedicated graph database — and it is rebuilt by an orchestrator when the coaching services start up. It is a living structure: as the ExperienceBank (§5.2) accumulates entries, new edges are automatically proposed to integrate the player's specific experience with the public knowledge. Multi-hop is what distinguishes a mature coach from a young coach: a young coach knows individual plays; a mature coach sees *game structures*. The graph encodes exactly these structures, and the coach gradually learns to navigate them — at first it jumps from one node to another in an uncoordinated way, after many demos it follows coherent patterns that surface high-level strategic advice.

Compare with: §5.1 (The Library — the graph navigated by FAISS), §5.2 (The Diary — how edges are added to the graph).

§6 — Training and Orchestration

§6.1 The Coach's School

Training the coach is a **school with a 4-phase curriculum**. Phase 1 is **kindergarten** (CALIBRATING, 0-49 demos ingested): the coach watches matches as a child watches cooking documentaries — absorbing patterns without cooking. Phase 2 is **middle school** (LEARNING, 50-199 demos): it moves from documentaries to cooking school, follows simple recipes under supervision. Phase 3 is the **vocational high school** (MATURING, transition): it cooks for friends, receives feedback, learns from its own mistakes. Phase 4 is the **diploma** (MATURE, 200+ demos): it becomes a master chef capable of inventing new dishes. The **CoachTrainingManager** is the **principal** of the school: it decides which model can study what and when, it tracks grades through the Observatory (§1.7), and scales the confidence multiplier of advice based on the current phase ($0.5\times \rightarrow 0.8\times \rightarrow 1.0\times$). Every model has its own curriculum: JEPA studies self-supervised pre-training with context/target split and InfoNCE; VL-JEPA learns the 16 concepts of coaching as sixteen school subjects (positioning, utility, economy, engagement, decision, psychology); RAP Coach learns the *pedagogy* with a composite loss. The **Teacher daemon** updates the curriculum when the library grows by 10% — the principal does not hold the student back a year if there aren't enough new examples, because repeating without novelty means overfitting, and overfitting means the coach learns the matches by heart instead of learning their principles. The 4-phase scale is calibrated empirically: a model with 30

demos has seen too little variety to judge a new player with confidence; at 100 demos it begins to recognize typical patterns; at 200 demos its internal calibration is stable. Pushing the coach into production too early (below 50 demos) produces "black box" recommendations that can confuse rather than help; waiting too long (beyond 400 demos) leads to diminishing returns. The scale is an engineering choice of prudence.

Compare with: §1.7 (The Report Card — how academic progress is measured), §6.2 (The Orchestra — who executes the curriculum).

§6.2 The Training Orchestra

The **TrainingOrchestrator** is the **conductor** who leads the models through epochs, validation, early stopping, and checkpointing. Every model is an instrument: JEPA is the first violin (representational pre-training, **JEPATrainer**), VL-JEPA is the second violin (conceptual alignment), RAP Coach is the chamber orchestra (multi-task composite loss, **RAPTrainer**), NeuralRoleHead is the percussionist (rapid classification). The conductor does not play instruments — they conduct: they set the tempo (learning rate schedule), signal changes of movement (early stopping), and when needed they call back a musician for a solo (focal training on a specific metric). The **callbacks** are like **sensors on a race car**: they collect data without interfering with the drive. The TensorBoard callback logs loss, accuracy, gradient norms, attention patterns; the ConvictionCallback updates the five signals of the Observatory; the DriftMonitor watches the z-scores of feature distributions and raises an alarm when the drift exceeds 2.5 standard deviations — *"the meta has changed, it's time to retrain"*. Among callbacks an iron rule reigns: **do not interfere with training**. If a callback breaks, training must continue; if a callback is slow, it cannot slow down the epochs. Every callback is an independent observer, like the 50 cameras filming a concert but none of which can stop the music. The reason callbacks are sacred is that they are the source of self-diagnostics: without them, a diverging training is discovered only after hours of wasted GPU. With them, training stops at the first significant irregularity. The price of this safety is the computational overhead — every callback steals a fraction of time — but the gain is priceless.

Compare with: §1.7 (The Report Card — how to read the sensor data), §6.3 (The Exam — the specific metrics the orchestra performs).

§6.3 The Exam with Answers on the Back

The **loss functions** of training are like **exam papers**, and each type of exam measures a different thing. **MSE** (Mean Squared Error) is a numerical math problem: "what is the exact value?". **Cross-Entropy** is a multiple-choice quiz: "what is the correct category among K?". **BCE** (Binary Cross-Entropy) is a series of yes/no questions. **InfoNCE** is an identification test within a group: "among 32 people, who is the right match for this photo?" — the τ (temperature) regulates how *sharp* the recognition lenses are (small τ = precise but fragile lenses; large τ = blurry but tolerant lenses). **KL Divergence** measures how much two probability distributions resemble the target one. **L1 Sparsity** is the constraint "use as few neurons as possible" (§1.3). **VICReg** keeps the class varied: without VICReg, all students give the same answer; with VICReg, the answers must be different (high variance) and uncorrelated (low covariance). The deadly sin of

training is **label leakage** — an exam with the answers printed on the back of the sheet: if `round_won` is in your 25 features (invariant `P-RSB-03`), the model learns to read the back of the sheet instead of studying, and in production, when the back isn't there, it no longer knows how to answer. This is why `round_won` is forbidden in training features. The hunt for label leakage is an almost religious discipline: every time a new feature is introduced, one must ask *"can it be computed only after the end of the round?"*. If the answer is yes, it is leakage. Subtle examples: `final_kda` (available only at round end), `round_money_earned` (includes the win bonus), `time_since_last_defuse` (only if the bomb was planted, i.e. the round is nearly over). Every feature passes an audit before entering the 25-dimensional vector.

Compare with: §6.2 (The Orchestra — who administers the exams), §1.7 (The Report Card — where the grades are written).

§6.4 The Talent Scout of Roles

The `RoleClassifier` is a **talent scout** who watches a player for 10 rounds and says *"this one is an AWPPer"*, *"this one is an entry"*, *"this one is an anchor"*, *"this one is a support"*, *"this one is a lurker"*. But the talent scout respects one rule: **they do not decide until they have seen enough**. The

`RoleThresholdStore` is like a cautious doctor who refuses to diagnose a rare condition if they have only seen two patients — they need a minimum sample (typically 30 rounds of observation) before assigning a role with confidence > 0.7. Until the threshold is reached, the role stays UNKNOWN and coaching adapts: generic advice instead of role-specific advice. When the sample grows, the `NeuralRoleHead` comes into play: a 5-output MLP that runs a **flash quiz** based on the 25 aggregated features of the player. The quiz and the classical talent scout work in consensus: if they agree, the classification is accepted; if they diverge significantly, the coach signals *"ambiguous role, the player is exploring"*. The **cold-start protection** is there because a schoolteacher who does not yet know the students must not label them on day one — and neither must the coach. The scout + quiz duality is wise: the classical scout is based on heuristic rules written by humans (*"did they use the AWP in 70% of rounds? they are an AWPPer"*); the neural quiz is based on patterns learned from data. Heuristic rules are transparent but rigid; the neural network is flexible but opaque. Using them together, the coach has a system that is transparent where the rules converge (*"both say AWPPer"*) and attentive to uncertainty where they diverge (*"the scout says entry, the network says support — let's explore"*).

Compare with: §4.5 (The Poker Face — another measure of "who you are" but on the axis of unpredictability), §4.2 (The Detective — roles are the context of attribution).

§7 — Analysis Engines

§7.1 The Squad of Detectives

The `AnalysisOrchestrator` coordinates a **squad of eleven specialized detectives**, each one with their own notebook of investigative techniques. The Statistical Detective computes HLTV 2.0 rating (see §9 of Book 2: kills, deaths, ADR, KAST%, impact, survival, flash assists). The Momentum Detective follows round-by-round trends (§7.2). The Engagement Distance Detective studies at what range the player engages most often (close/medium/long) and compares it with the pro baseline for their role. The Utility Detective checks whether grenades are being spent efficiently (§7.3 adjacent). The Win Probability Detective estimates round-by-round with an integrated Elo network (§4.4 nearby). The Entropy Detective measures the surprise of decisions. The Deception Detective (§4.5) and the Economy Detective complete the team. Eleven detectives share a code: **every investigation is autonomous**, none asks another for intermediate data, all read from the `MatchDatabase` and write into separate tables. This decoupling is crucial: if one engine breaks, the others continue; if one engine improves, the others do not need to be re-tested. The orchestrator collects the conclusions of all of them and hands them to the `CoachingService` (§4.1), which decides which insight to bring to the foreground based on the context of the advice. The choice of eleven detectives rather than a single "super-engine" is architectural. A single engine that does everything would be a massive monolithic model hard to maintain, debug, replace. Eleven specialized engines are like eleven experts each with one specialization: each one can be upgraded without touching the others, each one can be tested in isolation, each one can tell its story in a way that is understandable. It is the same reason hospitals have separate departments rather than a single "specialist of the human soul".

Compare with: §4.4 (The Chess Tree — a specific detective for strategic probability), §7.2 (The Mood Ring — a specific detective for momentum).

§7.2 The Mood Ring

The `MomentumTracker` is like a **mood ring** that changes color based on how the match is going. Every round has a *momentum score* computed from combined signals: time remaining at round close, number of consecutive kills without losses, survivors from the previous round, equipment differential. The tracker holds a sliding window of five rounds and produces discrete states — ICE (frozen momentum, negative spiral), COOL (control), WARM (good rhythm), HOT (positive streak), VOLCANIC (highlight-winning round). The ring cools with a constant decay: if you don't win rounds for a while, you naturally go back toward COOL regardless of how HOT you were before. Integrated with the ring there is a **heatmap** that visualizes where pro players tend to stand in the same situation — if your heatmap differs significantly from the pro baseline, the coach flags "*non-standard positioning, here is the pro reference*". The heatmap uses 128×128 cells for each CS2 map with pixel-perfect calibration (see the mapping section in the root README). Together, momentum + heatmap tell the *emotional temperature* of the round and the geography of winning positions, two pieces that the coach combines into feedback like "*you are HOT but playing on off-meta positions — consolidate before pushing*". Momentum is a signal that human players intuitively feel but

do not quantify. The mood ring makes the intuition visible: in a chart, the player sees exactly the moment at which their rhythm changed, and can correlate it with a specific choice. This visibility is formative: it helps build emotional awareness, a meta-skill that later translates into better decisions.

Compare with: §2.3 (The Three Lenses — momentum is a macro lens), §4.5 (The Poker Face — another measure of how you sound to the enemy).

§7.3 The Driving Instructor

The **BlindSpotDetector** and the **MovementQualityAnalyzer** together form the **coach's driving instructor**. The driving instructor notices when you forget to check a mirror, when you accelerate on a wet road, when you brake late into a corner. Likewise, the **BlindSpotDetector** records the angles that the player has consistently neglected to check during the previous 10 rounds on a given map — *"on Mirage mid, you always check window but always forget triple box"* — and raises a visual alert on the tactical map with a yellow circle over the blind angle. The **MovementQualityAnalyzer** is the instructor who watches replays in slow-motion: clean or rough strafe jumping? accurate counter-strafting before shooting or sliding while pressing? shoulder-first or chest-first peek? Every technique has a metric from 0 to 100 based on the difference against the standard movement of pros, and the analyzer returns a technical report with improvement priorities. The instructor never yells — they detect the pattern and communicate it calmly: *"in the last 20 matches, 12 deaths occurred during peeks with unfinalized strafe. I recommend 15 minutes of counter-strafe drill before the next match"*. Pedagogy is not anger — it is precise diagnosis followed by a targeted exercise. Movement in CS2 is one of the subtlest layers of the game — a player can have excellent aim and perfect strategic decisions but lose duels because of movement micro-defects that they don't consciously perceive. The driving instructor makes these micro-defects visible and turns them into concrete training goals. After a few weeks of targeted drill, the player literally feels the difference: the duels they used to lose because of a 50 ms lag, they now win; the counter-strafe becomes automatic; the peek synchronizes with the crosshair.

Compare with: §7.1 (The Squad — the instructor is one of the eleven detectives), §4.2 (The Detective of 5 Errors — the five categories the instructor diagnoses).

§8 — Data and Ingestion

§8.1 The Post Office of the Letter

The ingestion pipeline is a **post office** where every CS2 demo is a letter that goes through seven stages: **(1) Discovery** — the Scanner daemon finds the **.dem** file in a configured folder; **(2) Validation** — the **IntegrityChecker** verifies the file passes **MIN_DEMO_SIZE** (10 MB, invariant DS-12) and is a valid CS2 demo; **(3) Registration** — the demo is registered in the **DemoRegistry** with MD5 hash for deduplication; **(4) Parsing** — the **DemoParser** extracts every tick via demoparser2 into a per-match SQLite database; **(5) Feature Extraction** — the **FeatureExtractor** applies the 25-dimensional contract (§3.2); **(6) Stat**

Calculation — the `Analyzer` computes HLTV 2.0, round stats, economy; **(7) Training Queue** — if the 10% growth threshold is hit, the Teacher daemon is notified. The *post office* has a supervisor — `run_ingestion.py` — who coordinates the seven stages, handles errors (corrupt demos, files too small, duplicates) and produces daily statistics. Every stage has a rule: **you cannot skip**. A letter without validation is a risky letter; a letter without registration is one that can be processed twice; a parsing without feature extraction is wasted effort. The `run_worker.py` is the **backup postman**: it picks up stale tasks (> 5 minutes in the queue) from another process that got interrupted, preventing a demo from ending up half-processed forever. The sequentiality of the 7 stages is a robustness choice: every stage is a checkpoint, and if the system crashes between two stages, the demo resumes from the last completed one instead of starting over. This is critical because parsing a demo (stage 4) can take several minutes for longer demos — restarting from scratch after a crash would produce infinite failure cycles.

Compare with: §8.2 (The Customs Officer — the quality control of stage 2), §6.1 (The School — stage 7 seen from the school's side).

§8.2 The Demo Customs Officer

The `DemoFormatAdapter` is the **customs officer** who inspects incoming packages. Every CS2 demo that arrives at the post office (§8.1) is inspected: size > 10 MB (`MIN_DEMO_SIZE` , `demo_format_adapter.py:49` , invariant DS-12 — a legitimate pro demo is typically 300-850 MB, under 10 MB definitely means corrupt or incomplete), correct protocol version, CS2 signature present. The `DemoQualityScorer` is the **quality inspector** who performs a deeper check after the customs officer: does the demo have 15 rounds or is it a Short match? Are the player names readable? Is the server tick rate 128? Is the map in the Active Duty pool (7 maps: anubis, ancient, dust2, inferno, mirage, nuke, overpass)? Based on these inspections, every demo receives a *quality score* between 0 and 1. Demos with score < 0.6 are flagged for manual review; demos with score < 0.3 are rejected. The customs officer is merciless because a single corrupt demo can poison training: if you include a demo with the wrong tick rate or a half-corrupt demo, the model learns false patterns. The lesson of the customs officer is strict: better 99 excellent demos than 100 demos of which one is ruined. This principle is counterintuitive — "*more data = better*" is an ML mantra, no? But in the coaching context the principle does not apply: a model trained on low-quality demos produces low-quality advice, regardless of quantity. The customs officer is the first line of defense against silent quality degeneration, and the coach invests in this rigor because it reaps the benefits downstream, in the form of more accurate advice.

Compare with: §8.1 (The Post Office — where the customs officer lives), §9.2 (The Vault — where the demo ends up after the check).

§8.3 The HLTV Sports Chronicler

The `DemoParser` is an **experienced sports chronicler** who, watching a demo, compiles a detailed tick-by-tick report card: who killed whom, with what weapon, at what distance, after stealing the flash thrown by whom, how many seconds before the bomb was planted. The chronicler does not invent — they report. They do not interpret — they record. Interpretation comes from the analysis engines (§7.1). The HLTV

module (`hltv_sync_service.py` , `fetch_hltv_stats.py` if present) is the **spy team** — well-organized, discreet, rule-respecting — that collects pro statistics from HLTV sites. It uses Playwright for the headless browser, FlareSolverr in Docker to bypass Cloudflare, a `RateLimiter` to respect the site's limits, a `CircuitBreaker` (§9.2 adjacent) that opens the circuit if HLTV responds with consecutive errors and retries with exponential backoff. The result is the `hltv_metadata.db` database with three tables: `ProPlayer` , `ProTeam` , `ProPlayerStatCard` (161 pro players, 32 teams, 156 stat cards at the time of writing). The sync service lives in a separate process from the session engine daemons, to avoid WAL contention on the monolith database. Respect for HLTV's site rules is deliberate — the coach does not behave like an aggressive bot, does not do intense scraping, does not try to bypass protection systems beyond FlareSolverr (needed to legitimately get around Cloudflare). The philosophy is to be a good neighbor of the CS2 community: HLTV provides data for the community, and the coach uses a limited portion of it for comparison with the pros, without stressing them.

Compare with: §3.1 (The 5 Senses — the coach's sixth sense), §9.1 (The Archive — where the chronicler files their reports).

§8.4 The Deli Counter Ticket

At the deli you take a number. Not because the shopkeeper is bureaucratic, but because without a number two clerks serve the same customer and nobody serves the one behind. Ingestion has exactly this problem: **six** different doors can start it — the Home screen, the settings, the console command, batch ingestion, professional-demo ingestion, and the background worker. For a long stretch each of these doors looked at the queue, saw a waiting job and took it; if two doors looked at the same instant they saw **the same snapshot**, and the same demo was parsed twice, writing duplicate statistics that nobody could later tell apart from the good ones. The fix was not to post a supervisor to coordinate the clerks — that would have been a second mechanism to maintain, and to get wrong. It was to change the gesture: instead of *"I read that it is free, then I take it"*, the runner tells the database *"assign it to me **only if** it is still free"*, all in a single statement the database cannot split in half. The winner gets a confirmation, the loser gets a silent zero and moves on to the next job without complaint. This is the difference between managing a collision and **making it impossible**, and it is almost always the better choice: a problem that cannot occur needs no monitoring, no log line explaining it, and no memory from whoever maintains the code three years later. One courtesy remains: if you take a ticket and then give up, the number must go back into circulation — otherwise that job stays forever in an *"in progress"* state that nobody is progressing.

Compare with: §8.1 (The Post Office — the queue the ticket is taken from), §9.1 (The 3-Tier Archive — where the statistics that must not be duplicated end up).

§9 — Database and Storage

§9.1 The 3-Tier Archive

The storage system is a **three-tier archive**, each tier designed to avoid a different type of problem. **Tier 1** is `database.db`, the main monolith (18 SQLAlchemy tables defined in `database.py:_MONOLITH_TABLES` lines 54-73): player statistics, coach state, ingestion tasks, coaching insights, RAG knowledge, COPER experience bank, calibrations. **Tier 2** is `hlvtv_metadata.db` (7 tables: `ProPlayer`, `ProTeam`, `ProPlayerStatCard`), separated from the monolith because it is written by an external process (HLTV sync service) and the separation eliminates WAL contention between that process and the session engine daemons. **Tier 3** is `match_data/{id}.db`: a dedicated SQLite database for every match, containing tick-by-tick data (~100,000 rows per match) — three tables in every DB: `MatchTickState:110`, `MatchEventState:190`, `MatchMetadata:242` in `match_data_manager.py`. This separation solves the *Telemetry Cliff*: it prevents the monolith from growing indefinitely with high-frequency telemetry. The **WAL mode** (Write-Ahead Logging) is the revolving door of the archive: concurrent readers and writers enter without bumping into each other, because writes happen in a separate file (.wal) that is periodically consolidated. Total: 24 SQLAlchemy tables distributed across 3 tiers, with 5 PRAGMAs (`database.py:110-118`): WAL on, synchronous NORMAL, foreign_keys ON (DB-06), wal_autocheckpoint 512 (DB-07), busy_timeout 5000. The 3-tier architecture is not just engineering — it is a philosophy of data. Different data have different needs: the monolith has hot, low-frequency data (player stats that change rarely), HLTV has external data refreshed weekly, per-match telemetry has huge data that is "archivable" after analysis. Putting them together would create contention, slowdowns, and make the monolith file explode. Separating them allows each tier to optimize independently.

Compare with: §8.1 (The Post Office — stage 3 sends the demos here), §9.2 (The Vault — who protects the archive).

§9.2 The Backup Vault

The `BackupManager` is a **bank vault** — an automatic system that copies every database to a safe place on a scheduled cadence without ever interrupting the main archive. It uses the SQLite **Online Backup API** (`sqlite3.Connection.backup()`, `backup_manager.py:81-89`, invariant BE-01/DB-03) instead of `VACUUM INTO`, because Online Backup does not require exclusive locks and does not block live writers — it is like photocopying a book without removing it from the shelf. The retention policy is "1 latest + 7 daily + 4 weekly": the most recent backup is always kept, then the last 7 daily backups and the last 4 weeklies (consolidated weekly). Backups older than the oldest weekly are deleted, to prevent the vault from exploding. Alongside the BackupManager, the **fault tolerance** system is nuclear safety in multiple layers: (1) pre-commit hooks block code that violates invariants; (2) the Pulse daemon restarts a daemon that has stopped; (3) the HLTV scrapers' `CircuitBreaker` avoids hammering a site that responds with errors (states: CLOSED/OPEN/HALF_OPEN); (4) filesystem singleton locks prevent two instances of the app from writing over each other. The principle is the same as nuclear power plants: one layer can fail, two layers

can fail, but for the system to give in all layers must fail simultaneously. The vault has another function beyond restore: it is a **time machine** for debugging. When one suspects a state has been corrupted — for example a pro stat card looks wrong — one can open a backup from 3 days earlier and check whether the corrupted value was already there. This allows tracing subtle bugs that would otherwise be invisible.

Compare with: §9.1 (The Archive — what the vault protects), §6.2 (The Orchestra — another layered system).

§10 — System Overview

§10.1 The Robot that Watches Video

Imagine you have a **super-smart robot coach** who watches your CS2 matches as video recordings. First, it **watches** hundreds of professional matches and yours, taking tick-by-tick notes (this is ingestion, handled by the Scanner daemon and the Digester). Then, it **studies** the notes and learns what great players do differently from beginners, like a student advancing through school (CALIBRATING = kindergarten, LEARNING = middle school, MATURE = diploma) — this is done by the Teacher daemon in the background. When the time comes to give you advice, it does not guess: it consults its **notebook of tips** (RAG), the **memory of past sessions** (COPER) and what the **pros** would do in your exact situation, picking the source it trusts most based on the maturity of the models. Finally, it **explains** why: not just "*do this*", but "*do this **because** you keep getting caught off guard by mid control on Mirage*". It is like having a coach who has seen every pro match ever played, remembers every training session you have done, and can explain to you exactly why you should change strategy. Meanwhile, the Qt/PySide6 interface shows you everything in real time: 2D tactical map with your optimal ghost, radar charts of your skills, and a dashboard that tells you where your coach stands in its learning process. The Pulse daemon makes sure the system is always awake. This description is the high synthesis of the whole system — the metaphor from which all the others descend. It is the robot that watches video which, when put into action, reveals every department, every model, every analysis engine. The other three chapters of this section decompose the robot into different views — urban infrastructure, layered architecture — but the founding metaphor remains this one: a silent, tireless coach, who watches, studies, remembers, and explains.

Compare with: §10.2 (The City — the same overview from the infrastructure's point of view), §1.1 (The Factory — the same overview from the organizational point of view).

§10.2 The Coach's City

From the infrastructure point of view, the whole system is a **small self-sufficient city**. The **town hall** is the `SessionEngine` (`core/session_engine.py`) which coordinates the four city daemons — Scanner, Digester, Teacher, Pulse — like four aldermen with distinct responsibilities. The **power plant** (session engine at the lowest level) has four reactors that produce energy for all the other services; if one goes offline, the other three keep going and Pulse sends an alarm. The **civic archive** (storage, §9.1) is three tiers

below the town hall. The **public library** (knowledge, §5.1) and the **mayor's secret diary** (COPER, §5.2) are in the same wing but with different permissions. The **school** (training, §6.1) forms the adult citizens, the models. The **hospital** (coaching services, §4.1) cures the citizens who come for advice. The **police station** (RASP guard, [observability/rasp.py](#)) verifies the integrity of the buildings through SHA-256 hashes. The **post office** (ingestion, §8.1) distributes the demos arriving. The **sports chronicler** (HLTV sync, §8.3) lives just outside the city near the stadium to get the latest news. Every building has a door — an API endpoint or a public method — and no building knocks on the side doors of the others: everything goes through the square (central services). The city is small but complete: it can function without internet, without HLTV, without Ollama — losing functions but not collapsing. The city metaphor highlights a critical quality of the system: **self-sufficiency**. The coach was designed to work offline, on modest hardware, without cloud dependencies. This choice is not only technical but philosophical — the coach must belong to the player, not to someone else's servers. A player must be able to run the coach on their own PC without asking permission from anyone, without paying subscriptions, without anyone watching their data. The coach's city is a sovereign city.

Compare with: §10.1 (The Robot — the city seen from the user's side), §10.3 (The Dashboard — the control panel of the city).

§10.3 The Five-Layer Dashboard

The Qt/PySide6 desktop interface is the **dashboard of a modern sports car**: everything the driver (the player) needs to know is visible at a glance, every gauge has a precise meaning, and under the dashboard there is a **CAN bus** of signals and slots — Qt's Signal/Slot pattern — that lets 15 screens talk to 7 ViewModels without tight coupling. The overall architecture is a **five-layer cake**: (1) Qt UI on top ([apps/qt_app/screens/](#)), (2) MVVM ViewModels exposing observable properties, (3) Coaching and analysis Services (backend/services + backend/analysis), (4) Neural models and pure logic (backend/nn + backend/processing + backend/coaching), (5) Storage (backend/storage) at the bottom. The boundaries between layers are crossed only in one direction — UI calls ViewModel, ViewModel calls Service, Service calls Model or Storage — and never the other way around, to avoid circular dependencies that would turn the cake into a spaghetti plastic model. The project's **toolbox** contains PyTorch (neural networks), ncps (LTC), hopfield-layers (Hopfield), demoparser2 (Rust-based demo parsing), PySide6 (primary UI), SQLAlchemy+SQLModel (ORM), Alembic (migrations), Playwright (HLTV scraping), Rich (console TUI), FastAPI (internal API), Pydantic (validation). Every tool is a specialized craftsman: you use the right one for each job, never a hammer for a screw. The five-layer cake is one of the most valuable architectural invariants of the project — it is what makes the codebase maintainable despite having surpassed 120,000 lines of Python. Violating the directionality of the flows (for example, letting the model call a ViewModel) would produce a code smell known as *callback hell* or *architectural spaghetti*, and maintenance would become a nightmare. The discipline of the five layers, applied rigorously, is the reason the coach can be extended over time without collapsing under its own weight.

Compare with: §10.2 (The City — the infrastructure the dashboard commands), §1.1 (The Factory — the departments the 5-layer cake encodes).

§10.4 The Single Palette

A theatre staging three different shows can paint three complete backdrops, or it can paint **one** and change the lighting. The first road is more intuitive and is paid for in maintenance: when the director decides the red should shift toward orange, someone has to retouch three backdrops, and sooner or later forgets one — so the third show keeps the old red and nobody notices until opening night. For years the coach had three backdrops: three complete stylesheets, one per theme, with the colours written inside them. Today it has one — a single stylesheet template with **holes** where the colours go — and three palettes whose only job is to say which colour fills which hole. The gain is not elegance: it is that a colour now **exists in exactly one place**. The same palette that fills the stylesheet also fills the window's system palette and the function that decides what colour to show a rating in — so it can no longer happen that the stylesheet says one thing and the palette another, which is the kind of inconsistency no test catches and every user sees. Adding a fourth theme stops being the repainting of a backdrop and becomes the filling-in of a colour list. One detail remains worth telling, because it is the sort of thing you only discover live: when the user switches theme, the widgets that paint themselves must find out. But the theme engine lives only as long as an application session, and a widget cannot subscribe to something that will be replaced — it would be like leaving your contact details with a doorman who goes off shift. What was needed was a stable address: a **porter's lodge** that stays put across shift changes, which widgets announce themselves to once and which notifies everyone when the lights change.

Compare with: §10.3 (The Dashboard — the surface this palette colours), §1.1 (The Factory — the single-source principle applied to departments).

Final note from the author

These 38 analogies do not exhaust the richness of the four coach books — they are the pedagogical synthesis of them. Where a technical book would write *"the SuperpositionLayer applies a contextual gating regulated through L1-sparsity"*, here we write *"the 256-channel mixer automatically lowers the useless volumes"*. Neither of the two formulations is more correct than the other — they are complementary. The first is indispensable for whoever writes the code; the second is indispensable for whoever wants to understand the intent.

If during the reading of a chapter you find yourself thinking *"ok, technically clear, but why this choice?"*, come back to this volume. If you find yourself thinking *"ok, metaphor clear, but how is it implemented?"*, return to the technical chapter. The two readings done together are full understanding.

One last thought. Writing good analogies is harder than writing technical documentation. A technical document has a single criterion of correctness (does it match the code, yes or no?); an analogy has many: does it respect pedagogy? is it memorable? is it correct where it counts? does it withstand the critique of technical readers? does it not create false expectations? Every entry here has been iterated multiple times to reach this balance. If you find that a metaphor does not hold — that it creates a mental model that is wrong with respect to the code — it is a bug, and it is the most important kind of bug to report, because distorted pedagogy is worse than silence.

End of the Book of Analogies (English translation). The Italian canonical version is in [analogy-book.md](#); the Portuguese translation in [analogy-book-pt.md](#), parity-checked against it.